Mini Assessment

Name	Boreddy Jyothi	**Reg No**	99230040012
Dept	CSE(DS)	**Date**	11-09-2026

Q1. If a database connection times out, what should your program do?

A) A connection timeout means the database server did not respond within the configured time. The program should handle the situation safely instead of crashing.

→ Catch the appropriate timeout/database exception and prevent the application from stopping unexpectedly.
→ Retry the connection a limited number of times, using a short backoff delay between attempts.
→ Set a sensible connection timeout so the program does not wait indefinitely.
→ If all retries fail, record the error in the logs and inform the user with a clear message such as “Database service is temporarily unavailable.”
→ The program should fail gracefully and allow the user to try the operation again.

Q2. If cursor.fetchone() returns None, what should your program do?

A) **cursor.fetchone()** returns **None** when the query executes successfully but no matching row is found. This should be treated as a normal “record not found” situation.

→ Check whether the returned value is None before trying to access its fields.
→ Return a clear “record not found” response or an appropriate default value.
→ Do not index, unpack, or access attributes of None, because this can cause an exception.
→ Log useful query or request context if the missing record needs investigation.
→ Continue the program normally rather than treating the absence of a row as a database failure.

Q3. If a column value is missing from a query result row, why might .get() be safer than []?

A) When a database result is represented as a dictionary, using **row['column']** can raise a **KeyError** when that column is absent. In contrast, **row.get('column')** can safely return None or a supplied default value.

→ It avoids unnecessary KeyError exceptions for optional fields.
→ A default value can be supplied, for example: **row.get('email', 'Not available')**.
→ It is useful when different queries return different sets of columns.
→ The returned value can be checked before further processing.
→ This makes database-handling code safer and more reliable when fields are optional or dynamically selected.